Southern New Hampshire University

CS 300 Pseudocode Document

Example Function Signatures

Below is an example of a function signature that you can use as a guide to help address the program requirements using each data structure for the milestones. The pseudocode for finding and printing course information is also given below and depicted in bold to help you get started. The provided pseudocode is for a vector data structure, so you may use this pseudocode in your first milestone as is. The hash table and tree structures are also shown below. But these structures are left for you to do in future milestones.

```
//Vector - Milestone 1

void searchCourse(Vector<Course> courses, String courseNum) {
        for all courses
                if the course is the same as courseNum
                        print out the course information
                        for each prerequisite of the course
                                print the prerequisite course information
}
//Hash Table - Milestone 2

void searchCourse(HashTable<Course> courses, String courseNum) {

}

//Binary Search Tree – Milestone 3

void searchCourse(Tree<Course> courses, String courseNum) {
```

Example Runtime Analysis

When you are ready to analyze the runtime for the Project One data structures for which you created the pseudocode, use the example chart below to support your work. This particular example is for printing course information when using the vector data structure. As a reminder, this is the same pairing that was bolded in the pseudocode from the first part of this document. The example only covers the search function for the vector structure. You do not have to complete your runtime analysis until Project One. However, working on your analysis now may help you understand the changes as you complete the milestones. Don't forget to include your charts in Project One. You will submit Project One in Module Six.

Below is all of the updated Pseudocode. It should satisfy all requirements:

Milestone One (Vector Pseudocode):

Course loading Function (Vector):

```
void loadCourses(string filename) {
	if (!file.open(filename)) {
		print("Error! Could not open file!");
		return;
	}

	while (getline(file, line)) {
		vector<string> courseInfo = split(line, ',');
		if (courseInfo.size() < 1) {
			print("Error! Wrong data format!");
	}

	courseNum = courseInfo[0];
	courseTitle = courseInfo[1];

	vector<string> prerequisites(courseInfo.begin() + 2, courseInfo.end());

	Course course = createCourse(courseNum, courseTitle, prerequisites);
	courses.push_back(course);
	}

	file.close();
}
```

Search and Printing Function (Vector):

```
void searchCourse(Vector<Course> courses, string courseNum) {
	bool found = false;
	for (Course course in courses) {
		if (course.courseNum == courseNum) {
			print("Course: " + courseNum + " " + courseTitle);

		for (string prereq in course.prerequisites) {
			print("Prerequisite: " + prereq);
	}
		found = true;

	if (!found) print("Course not found.");
}
```

Printing courses in correct order Function (Vector):

```
void printCourseList(Vector<Course> courses) {

	sort(courses.begin(), courses.end(){
		return a.courseNum < b.courseNum;
	}

	for (Course course in courses) {
	print("Course: " + courseNum + " " + courseTitle);

	}
}
```

Our Menu (Vector):

```
void displayMenu() {
	bool whileRunning = true;

	while (whileRunning) {
		print("1: Load course data");
		print("2: Print course list");
		print("3: Search course by number");
		print("9: Exit");

		string choice = getInput();
		switch (choice) {
		case "1":
			loadCourses("courseInfo.txt");
			break;
		case "2":
			printCourseList(courses);
			break;
		case "3":
			print("Enter course number:");
			string courseNum = getInput();
			searchCourse(courses, courseNum);
			break;
		case "9":
			whileRunning = false;
			break;
	}
}
```

Vector Runtime Analysis Table:

Code	Line Cost	# Times Executes	Total Cost
for all courses	1	n	n
if the course is the same as courseNum	1	n	n
for each prerequisite of the course	1	1	1
for each prerequisite of the course	1	n	n
print the prerequisite course information	1	n	n
		Total Cost	4n + 1
		Runtime	O(n)

Milestone Two (Hash Table Pseudocode):

Course loading Function (Hash Table):

```
void loadCourses(string filename) {
        if (!file.open(filename)) {
                print("Error! Could not open file!");
                return;
        }

        while (getline(file, line)) {
                vector<string> courseInfo = split(line, ',');
                if (courseInfo.size() < 1) {
                        print("Error! Wrong data format!");
                }

        courseNum = courseInfo[0];
        courseTitle = courseInfo[1];

        vector<string> prerequisites(courseInfo.begin() + 2, courseInfo.end());
        Course course = createCourse(courseNum, courseTitle, prerequisites);
        hashTable[hash(course.courseNum)] = course;
        }
        file.close();
}
```

Search and Printing Function (Hash Table):

```
void searchCourse(HashTable<Course> courses, string courseNum) {
	int key = hash(courseNum);

	if (hashT[key] != nullptr) {
		print("Course: " + hashT[key].courseNum + " " + hashT[key].courseTitle);

	for (string prereq : hashT[key].prerequisites) {
		print("Prerequisite: " + prereq);
	}else {
	print("Course not found.");
	}
}
```

Printing courses in correct order Function (Hash Table):

```
void printCourseList(HashTable<Course> courses) {
	vector<string> keys = getKeys(hashT);
	sort(keys.begin(), keys.end());

	for (string key in keys) {
		print("Course: " + hashT[key].courseNum + " " + hashT[key].courseTitle);
	}
}
```

Our Menu (Hash Table):

```
void displayMenu() {
	bool whileRunning = true;

	while (whileRunning) {

		print("1: Load course data");
		print("2: Print course list");
		print("3: Search course by number");
		print("9: Exit");

		string choice = getInput();

		switch (choice) {
			case "1":
				loadCourses("courseInfo.txt");
				break;
			case "2":
```

```
                printCourseList(courses);
                break;
            case "3":
                print("Enter course number:");
                string courseNum = getInput();
                searchCourse(courses, courseNum);
                break;
            case "9":
                whileRunning = false;
                break;
        }
}
```

Hash Table Runtime Analysis Table:

Code	Line Cost	# Times Executes	Total Cost
for all courses	1	n	n
if the course is the same as courseNum	1	n	n
for each prerequisite of the course	1	1	1
for each prerequisite of the course	1	n	n
print the prerequisite course information	1	n	n
		Total Cost	4n
		Runtime	O(n)

Milestone Three (Binary Search Tree Pseudocode):

Course loading Function (BST):

```
Tree<Course> loadCourses(string filename) {
    if (!file.open(filename)) {
        print("Error! Could not open file!");
        return;
    }
```

```
        Tree<Course> treeCourses;
        string line;

        while (getline(file, line)) {
                vector<string> data = split(line, ',');

        if (data.size() < 1) {
                print("Error: Invalid format.");
        }

        Course course = createCourse(data);
        bool missedPrereq = false;

        for (string prereq in course.prerequisites) {
                if (!checkCourseInFile(prereq)) {
                        print("Error! Missing prerequisite for " + prereq + "!");
                        missedPrereq = true;
                }
        }

        if (!missedPrereq) {
                insertCourse(treeCourses, course);
                }
        }
        return treeCourses;
}
```

Search and Printing Function (BST):

```
void searchCourse(Tree<Course> treeCourses, string courseNum) {

        TreeNode<Course> *node = findCourse(treeCourses, courseNum);

                if (node != nullptr) {
                        print("CourseNum: " + node->getCourse().courseNum);
                        print("CourseTitle: " + node->getCourse().courseTitle);

                if (!node->getCourse().prerequisites.empty()) {
                        print("Prerequisites: ");
                for (string prereq : node->getCourse().prerequisites) {
                        print(prereq);
                } else {
                print("Course not found.");
        }
}
```

Our Menu (BST):

```
void displayMenu() {
        bool whileRunning = true;
        while (whileRunning) {
        print("1: Load course data");
        print("2: Print course list");
        print("3: Search course by number");
        print("9: Exit");

        string choice = getInput();
        switch (choice) {
        case "1":
                loadCourses("courseInfo.txt");
                break;
        case "2":
                printCourseList(treeCourses);
                break;
        case "3":
                print("Enter course number:");
                string courseNum = getInput();
                searchCourse(treeCourses, courseNum);
                break;
        case "9":
                whileRunning = false;
                break;
    }
}
```

Southern New Hampshire University

Binary Search Tree Runtime Analysis Table:

Code	Line Cost	# Times Executes	Total Cost
for all courses	1	n	n
if the course is the same as courseNum	1	n	n
for each prerequisite of the course	1	1	1
for each prerequisite of the course	1	n	n
print the prerequisite course information	1	n	n
		Total Cost	4n + log n
		Runtime	O(n)

Advantages and Disadvantages

Vectors:

One of the main advantages of vectors is that it's quite easy to implement, test and manage. With vectors we can easily sort with the functions provided to us and it also works well with linear operations. This is why it's so widely used and often one of the first learned data structures.

The main disadvantage compared to other data structures is mainly its speed. The lack of speed makes it less than ideal when working with large datasets and can take up huge amounts of time.

Hash Tables:

In contrast to vectors, Hash Tables are much faster, so much so that they are ideal for large datasets. This would be great to implement in a large dataset that may require us to do more than several searches.

The main problem with Hash Tables though, also in contrast to vectors, is how tricky they can be to use. They lack a specific order and we have to take care of the sorting ourselves. This can lead to a bunch of time and effort creating these processes, especially if we have to deal with collisions.

Binary Search Trees:

The Binary Search Tree is also quite fast, although not as fast as Hash Table's but much faster than vectors, they are also easy to manage due to their sorter order which may make some parts of the program easier, say printing courses.

The main issue with Binary Search Tree's are how complex they are. Yes, they can be easier to order than Hash Tables but overall they are a pretty complex data structure and improper handling could lead to significant performance issues.

My Recommendation:

After spending these few weeks working with these data structures I would have to say that for our Course program the best option is Hash Tables. I originally thought about BST's as they can be the best of both worlds but because of the complexity and nature of course searching I decided that Hash Tables are better. This is because Hash Tables are less complex of a data structure, extremely fast even faster than most well optimized BST's and very efficient. Unfortunately, Vectors are just too slow to be added in this case, I learned this in our previous assignments.